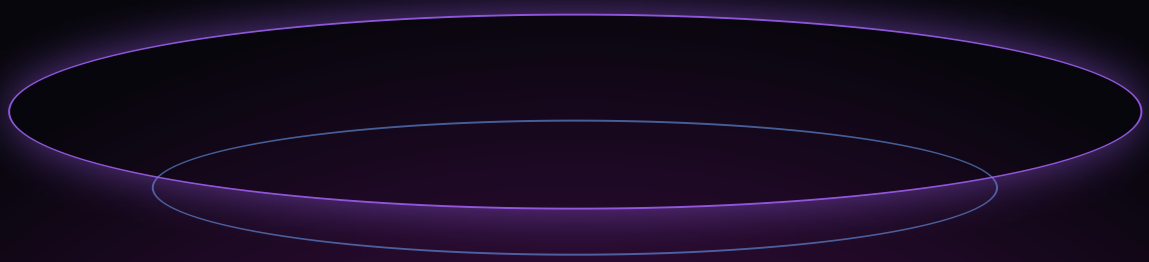
SIG REALITY SPECTRA · VIT PUNE

Website Technical Plan & Architecture

The proposed technology stack, system design, data model and delivery roadmap for the official website of the club's AR / VR / XR special interest group.



DOCUMENT	VERSION	DATE	STATUS
Technical proposal	1.1 · draft for review	September 2026	Awaiting committee approval

CONTENTS

What this document covers

01	Overview and goals
02	Technology stack
03	System architecture
04	Information architecture
05	Data and content model
06	Key flows
07	Rendering and caching
08	Security model
09	Experience layer: design, motion, 3D
10	Codebase structure and rules
11	Quality, testing and budgets
12	Environments and deployment
13	Delivery roadmap
14	Risks, decisions and open inputs

At a glance

6
public pages, managed through 5 admin tools

0
database servers to run or patch

< 10 s
from an admin's save to the live site, no redeploy

10
delivery stages, each independently reversible

The plan in one paragraph

A single **Next.js 16** application serves both the public site and a small admin panel from **Vercel**. The little copy that rarely changes — intro, mission, vision, domains and contact details — lives in Git as validated JSON; everything the committee edits week to week — events and their photos, team members, recruitment and sponsorship requests — lives in **Upstash Redis** and **Vercel Blob**, reached only through one repository layer. Every admin save expires exactly the cached pages it affects, so the live site updates within seconds without a rebuild. The visual layer is a dark "XR lab" built on design tokens, with one progressively enhanced 3D hero that falls back to a designed still on low-power devices.

VISITORS
See the current event and register for it, browse past events and photos, meet the team, and apply when recruitment is open. Never log in.

ADMINS
Two or three allowlisted Google accounts. Manage events and photos, team members, recruitment and sponsorship requests — from a phone if needed.

MAINTAINERS
Update the intro, mission, vision, domains and contact details by editing one JSON file, guided by copy-paste templates.

01 · OVERVIEW

Goals, constraints and scope

The website is the club's public face for recruitment, sponsorship and its permanent record of work. It has to look credible first and impressive second, and it has to outlive the committee that builds it.

Goals

Credible, cinematic presence

A dark, professional "XR lab" — a place that builds real things, not a gaming landing page — with one signature 3D moment.

Continuous updates without developers

Events, gallery photos, team members, recruitment and sponsorship requests are handled in an admin panel; the rest is a one-file Git edit.

Mobile-first and fast

Most recruitment traffic arrives from phones via QR codes and social posts. The site must stay smooth on a mid-range Android.

Hand-over-proof

Club-owned accounts, plain-language guides and yearly admin transfer by editing a single environment variable.

Hard constraints

CONSTRAINT	WHAT IT FORCES IN THE DESIGN
Admin changes are live immediately, with no redeploy	Tag-based cache invalidation on every write; admin data cannot live in the build output.
No traditional database to operate	Serverless key-value storage (Upstash Redis) and object storage (Vercel Blob), behind an interface so Postgres can replace them later.
Public visitors never authenticate	Public pages are static or cached; authentication exists only for `/admin` and is never linked publicly.
Nothing an admin types can break a public page	One shared validation schema per entity, fixed-ratio layouts, clamped text and an empty state for every list.
Accounts must belong to the club	GitHub org, Vercel, Upstash, Google Cloud, Cloudflare and the domain are all registered to club-owned logins.
Registrations and applications happen elsewhere	Events link out to their registration page (typically VIERP) and recruitment links out to a Google Form; the site never handles either.

Scope for version 1

IN SCOPE

- Six public pages: Home, About, Events, Gallery, Recruitment, Contact
- 3D landing hero with graceful fallbacks
- Admin tools: events with photos and a timeline, standalone gallery photos, team members, recruitment with a roles PDF, sponsorship requests
- Sponsorship form with spam protection and an admin inbox
- SEO, social previews, analytics with QR/UTM tracking
- Automated tests, CI, nightly backups and handover docs

DELIBERATELY OUT OF SCOPE

- Member accounts or any public login
- A custom recruitment database — Google Forms remains the application channel
- Projects, achievements and resources pages
- Separate pages per event, project or photo album
- Blog and knowledge base (v2 backlog)
- Handling registrations or payments

02 · TECHNOLOGY STACK

A locked, deliberately small stack

Every layer is chosen to remove something the club would otherwise have to operate. The stack is fixed for the duration of the build; any proposed substitution is raised and agreed, never made silently.

LAYER	CHOICE	WHY IT WAS CHOSEN
Framework	**Next.js 16 · App Router · TypeScript (strict)**	One codebase for the public site, admin panel and server logic. Server Components keep JavaScript off pages that do not need it.
Package manager	**pnpm**	Fast, disk-efficient, strict dependency resolution.
Styling	**Tailwind CSS v4 · shadcn/ui (admin only)**	Token-driven styling from a single `@theme`; shadcn/ui gives the admin panel accessible primitives without leaking into the public design.
3D	**React Three Fiber · drei**	Declarative three.js scenes for the AR/VR hero, with device-tier control.
Scroll animation	**GSAP · ScrollTrigger**	Frame-accurate, cinematic scroll choreography and pinning.
UI motion	**Motion**	Page transitions, hover states and animated list layouts.
Smooth scroll	**Lenis**	Smooth scrolling that shares one animation loop with GSAP.
Data	**Upstash Redis**	Serverless key-value store via the Vercel Marketplace; nothing to host or patch; instant, consistent writes.
Images	**Vercel Blob**	Event posters and photos, gallery and team photos, the recruitment PDF and nightly backups — uploaded straight from the browser.
Validation	**Zod**	One schema per entity, shared by the admin form, the server and the public renderer.

LAYER	CHOICE	WHY IT WAS CHOSEN
Admin auth	**Auth.js · Google · JWT sessions**	No user table and no passwords; access is an email allowlist in configuration.
Forms	**react-hook-form · Zod**	Type-safe forms with inline errors that match server rules exactly.
Spam protection	**Cloudflare Turnstile · honeypot · rate limit**	Invisible-to-humans bot protection on public forms.
Analytics	**Vercel Analytics**	Privacy-friendly traffic and UTM/QR campaign tracking without custom code.
Testing	**Playwright · Vitest**	End-to-end proof that admin changes reach the public site; unit tests for schemas and utilities.
Hosting & CI	**Vercel · GitHub Actions**	Zero-configuration deploys, preview environments per pull request, gated merges.

Component sourcing

Animated UI building blocks will be sourced from the **React Bits** registry through the shadcn CLI, always in the TypeScript + Tailwind variant. Vendored components are treated as starting points: each is reviewed for the project's motion, accessibility and token rules before it ships, and any component that introduces a new runtime library (for example the WebGL library `ogl`) needs explicit approval.

Platform notes that shape the design

Instant updates: `updateTag`

Next.js 16 provides read-your-writes cache invalidation from server actions — the mechanism behind "save and it is live".

Route guarding: `proxy.ts`

The request interceptor (formerly "middleware") protects `/admin` on the Node.js runtime.

Typed routes and params

Statically typed links make a dead internal link a build error; route parameters are asynchronous.

Cache Components

Tagged caching requires `cacheComponents`; pages prerender a static shell and request-time values stay out of the cache.

03 · SYSTEM ARCHITECTURE

One application, two storage services

Three kinds of people reach one Next.js application through Vercel's edge network. Everything that touches stored data goes through a single repository layer, which is the only code allowed to talk to Redis or Blob.

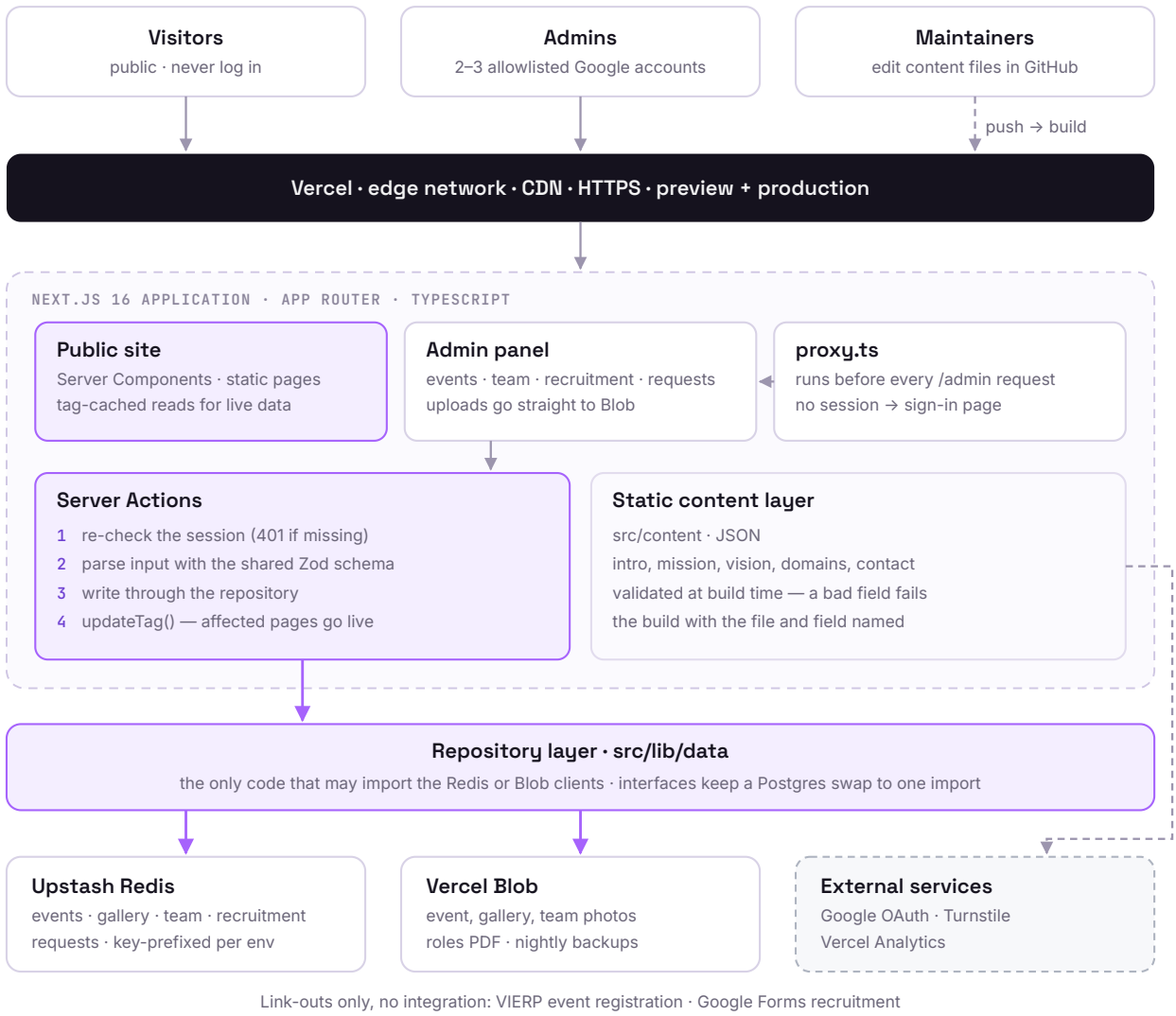


FIG 1 Container view. Solid violet paths are the only routes to stored data; dashed paths are external or build-time.

Public reads

Pages render on the server. Live data is fetched through cached repository calls tagged `events`, `team` and `recruitment`.

Admin writes

Server actions authenticate, validate, write, then expire only the tags they touched. No rebuild, no redeploy.

Content edits

A maintainer's JSON change is a Git commit; the resulting build validates every file before it can deploy.

04 · INFORMATION ARCHITECTURE

Six pages, five admin tools

A deliberately small public site with no per-record URLs: events, photos and team members render inline on the page that lists them. Five primary links and one call to action keep the navbar usable at 360 px.

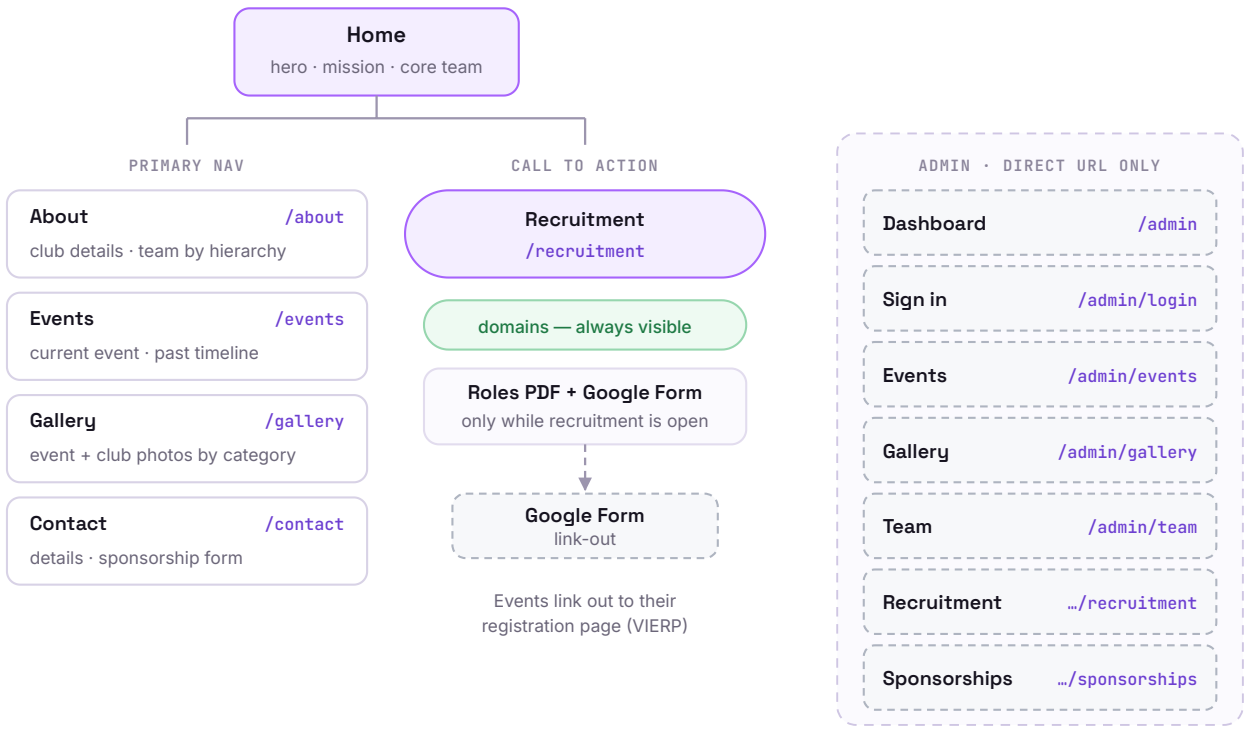


FIG 2 Sitemap. The admin area is never linked publicly, is `noindex`, and requires sign-in.

ADMIN TOOL	WHAT AN ADMIN DOES THERE
Events	Adds the current event with its details and registration link. When it ends it is kept for later; the admin adds a recap and photos and puts it on the timeline. Past events can also be added directly.
Gallery	Uploads standalone club photos with a category and alt text. Photos of events on the timeline appear in the gallery automatically.
Team	Adds members with name, position and photo, sets their place in the hierarchy, and marks core members, who also appear on the home page.
Recruitment	Opens or closes recruitment, uploads the roles and responsibilities PDF, and sets the Google Form link.
Sponsorships	Reads requests sent from the Contact page and marks them new, contacted or closed.

One source of truth for routes. A single configuration file lists every public page with its label, description and navigation placement. The navbar, footer, page metadata, `sitemap.xml` and the route tests all read from it, and a test fails if a per-record route is ever added without a design decision.

05 · DATA AND CONTENT MODEL

Two kinds of data, two homes

A handful of rarely changing copy lives in Git, where every change is reviewable. Everything the committee changes week to week lives in Redis and Blob, where an admin can change it instantly.

ENTITY	STORED IN	EDITED AT	CACHE TAG	READ BY
Site info, domains, contact details	Git (JSON)	a Git commit	— build time	Home, About, Recruitment, Contact — a malformed file fails the build
Event + photos	Redis + Blob	`/admin/events`	`events`	Events, Gallery
Gallery photo (standalone)	Redis + Blob	`/admin/gallery`	`gallery`	Gallery
Team member	Redis + Blob	`/admin/team`	`team`	Home (core only), About (everyone)
Recruitment settings	Redis + Blob	`/admin/recruitment`	`recruitment`	Recruitment, navigation call to action
Sponsorship request	Redis	`/contact` form	none	Admin inbox only — never cached

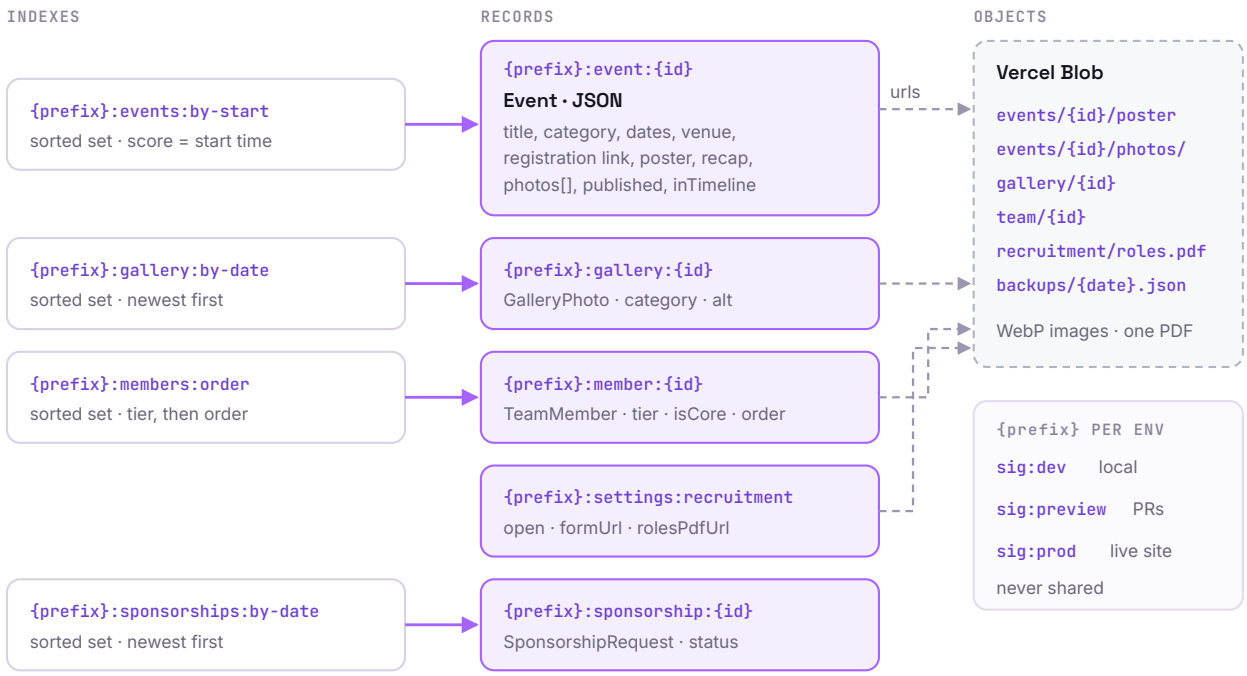


FIG 3 Redis key model. Multi-key writes run in one transaction so records and indexes can never drift apart.

Entity schemas (planned)

EVENT

id · title 3–80 · category workshop | hackathon | meetup | talk | competition | other · summary 20–160 · markdown description ≤ 5000 · start / optional end · venue · mode offline | online | hybrid · registration link (https) · poster + alt text · published · recap ≤ 2000 · up to 60 photos, each with alt text and dimensions · inTimeline.

GALLERY PHOTO

id · image with alt text (required) and dimensions · category, from the same list as events · optional caption. Shown with the photos of timeline events, grouped by category, newest first.

TEAM MEMBER

id · name · position · photo + alt text · tier (place in the hierarchy) · isCore (also shown on the home page) · order within the tier.

RECRUITMENT SETTINGS

open · Google Form link (https) · roles and responsibilities PDF. Recruitment can only be opened when both links are set; safe "closed" defaults are returned if no record exists.

SPONSORSHIP REQUEST

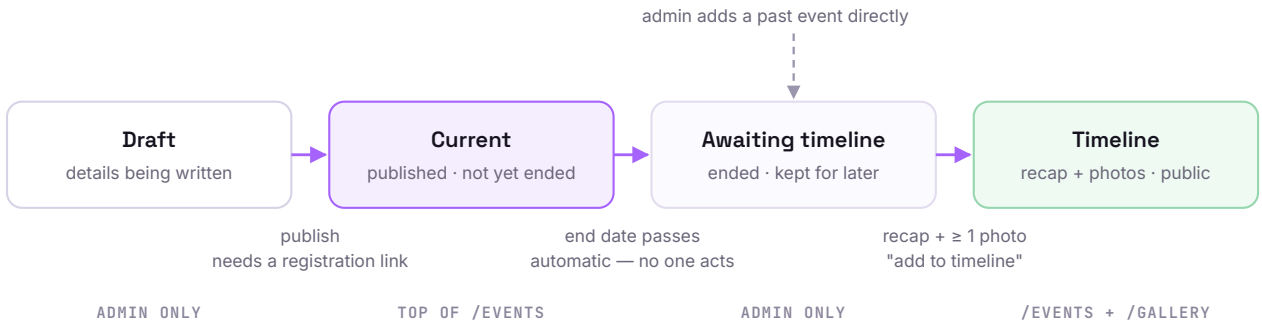
id · name · organisation · email · optional phone · type sponsorship | collaboration | other · message 20–2000 · status new | contacted | closed · created at.

Derived, not stored. An event's phase — current, awaiting timeline or on the timeline — is computed from its dates and flags when a page is requested. The gallery combines the photos of timeline events with standalone gallery photos, grouped by one shared category list. Updates carry an `updatedAt` check so two admins cannot silently overwrite each other, and deleting a record deletes its files.

06 · KEY FLOWS

An event's life, and save to live

Event lifecycle



Phases are computed when a page is requested, never stored. Deleting an event also deletes its poster and photos.

FIG 4 Event lifecycle. An ended event leaves the "current" slot on its own; the admin finishes it later.

Saving an event

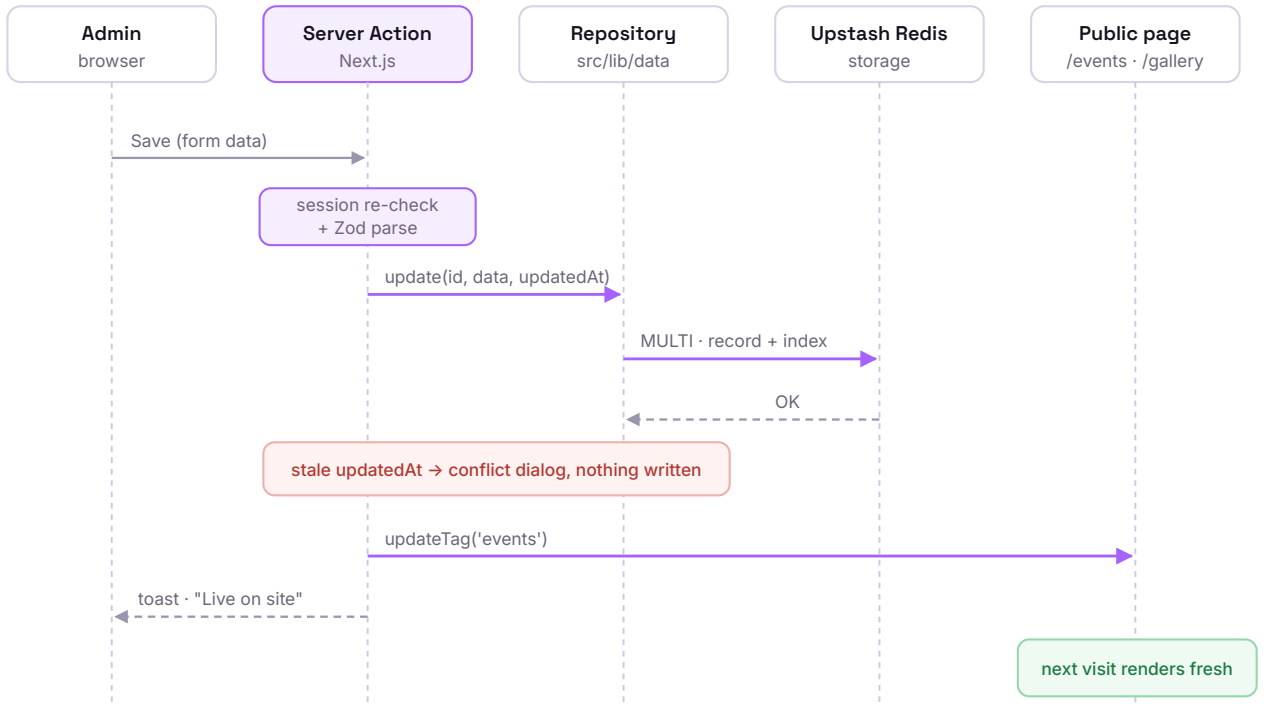


FIG 5 Save-to-live sequence. The target is under ten seconds from click to public page, with no rebuild.

The admin editor previews the event with **the same components the public site uses**, so what an admin approves is exactly what visitors see. Team and recruitment saves follow the same path with the `team` and `recruitment` tags.

Uploads and sponsorship requests

Uploading photos, posters and the roles PDF

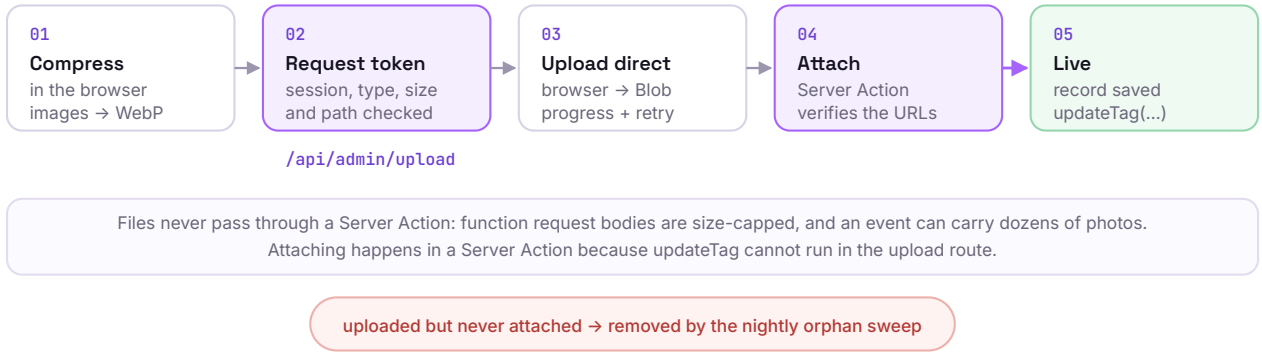


FIG 6 Browser-direct uploads. The token only allows one content type, one size limit and a path under the record being edited.

UPLOAD	BLOB PATH	RULES
Event poster	`events/{eventId}/poster-*.webp`	cropped to a fixed aspect ratio
Event photos	`events/{eventId}/photos/*.webp`	up to 60 per event, alt text required
Team photo	`team/{memberId}-*.webp`	square crop
Gallery photo	`gallery/{photoId}-*.webp`	alt text and category required
Roles PDF	`recruitment/roles-*.pdf`	PDF only; replacing it deletes the old file

Handling a sponsorship request

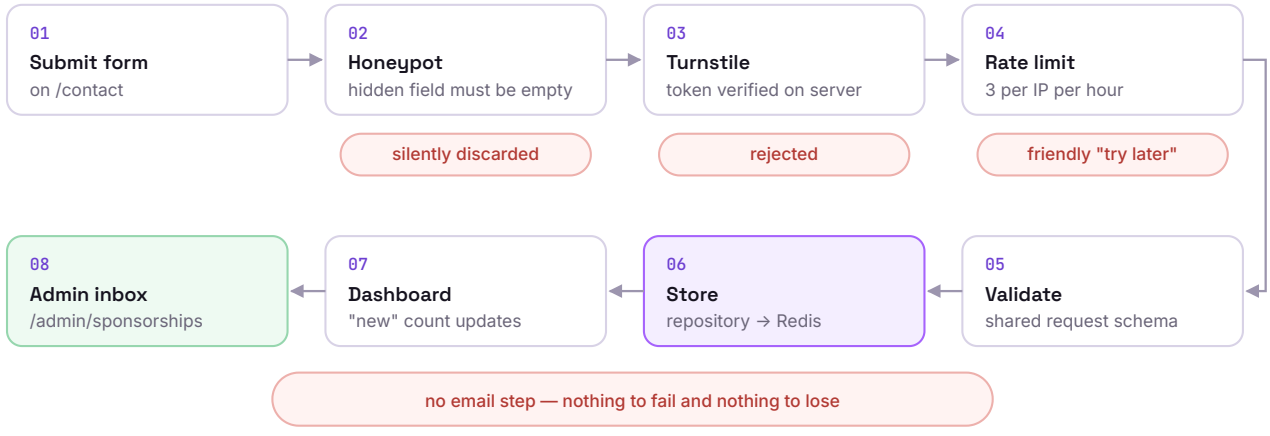


FIG 7 Sponsorship pipeline. Cheap checks run first; a valid request is stored and appears in the admin inbox.

07 · RENDERING AND CACHING

Static where possible, fresh where it matters

The application enables Next.js **Cache Components**: every page prerenders a static shell, public data is read through cached functions labelled with tags, and admin writes expire exactly the tags they touch. A write that does not expire its tags is an incomplete write.

PAGE	RENDERING	CACHE TAGS	NOTES
Home `/`	Static shell · Git content · cached core team	`team`	Hero, mission and vision are build-time content.
About	Static shell · Git content · cached team	`team`	Members grouped by tier, then ordered.
Events	Static shell · cached events · current/past split at request time	`events`	An event leaves the "current" slot when it ends, with no write and no rebuild.
Gallery	Static shell · cached timeline photos + standalone photos	`events` `gallery`	Loads more on request; category filter runs in the browser; images carry dimensions, so nothing shifts as they load.
Recruitment	Static shell · Git domains · cached settings	`recruitment`	PDF and form link rendered only while open.
Contact	Fully static · form posts to a Server Action	—	Requests are never cached or shown publicly.
Site navigation	Shared layout · cached settings	`recruitment`	The Recruitment call to action shows when applications are open.
Admin `/admin/*`	Dynamic per request, inside `Suspense`	—	Uncached reads, never indexed.
`sitemap.xml`	Static	—	Six fixed URLs; there are no per-record pages.

Why this works without a rebuild

READ
Public pages read events, team and recruitment through cached functions labelled with a tag.

WRITE
A Server Action saves the change, then calls `updateTag` for exactly the tag it affected.

RESULT
The next request re-renders only those pages with fresh data; everything else stays cached.

> **The current time is never cached.** Whether an event is current or past is decided in the page after `connection()`, outside any cached function — otherwise the first visitor's clock would be frozen into every later response.

08 · SECURITY MODEL

Defence in depth, with no passwords to lose

Access control is one environment variable. Each layer below assumes the one outside it has already failed.

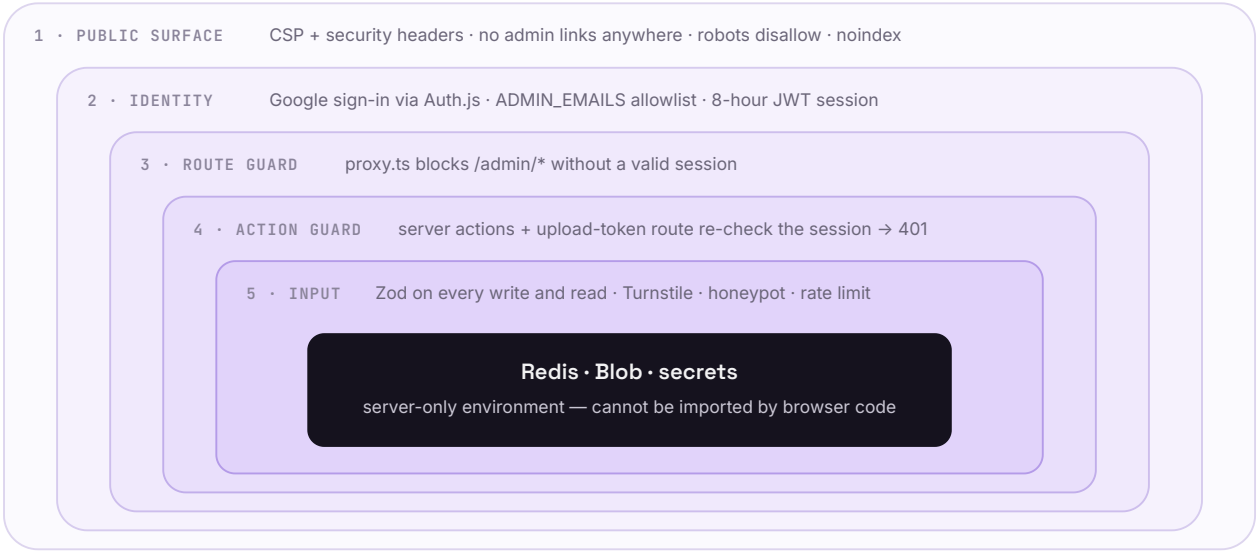


FIG 8 Security layers, outermost first.

Identity without a user table

Only Google accounts on the allowlist can sign in; everyone else sees a clean "not authorised" page. Comparison is case-insensitive and whitespace-trimmed. Yearly handover is editing `ADMIN_EMAILS`.

Secrets stay on the server

Environment variables are validated at startup; server secrets live in a module that cannot be imported by browser code, so a key cannot leak into the client bundle by accident.

Uploads and scheduled jobs

Upload tokens are issued only to an admin session, for one content type, size limit and path. The nightly job accepts only the `CRON_SECRET` bearer token. A CSP allows only Blob, Turnstile and analytics origins.

Enforced, not just documented

A lint rule blocks Redis/Blob imports outside the repository layer; end-to-end tests assert admin routes are `noindex` and that no public page links to them.

09 · EXPERIENCE LAYER

“Light is the interface”

A dark XR lab lit by a single beam from the logo's spectrum: magenta for the hottest highlight, violet for everything interactive, azure for the cool edge. Everything else is near-black and silver. Saturated colour never covers more than a tenth of any screen.

Spectrum — decoration and accents only

magenta	orchid	violet	indigo	azure	blue	cyan
#FB00EB	#E024F2	#A663FC	#635ACF	#6EA6FE	#3470C1	#7CD5FF

Base and text — lab darkness and logo silver

bg	surface	raised	border	text	body	muted
#07060C	#0E0D15	#16131F	#332A3A	#F2F2F3	#D8D8D7	#8E8A9A

display	**Space Grotesk**
body	Inter for all reading text, 16–18 px, never wider than 68 characters.
label	JETBRAINS MONO · VIT PUNE

THREE WAYS LIGHT MAY APPEAR

Spectra gradient — one headline word, the primary CTA border, active indicators.

Horizon glow — light rising below content, at most one per section.

Rim light — a 1 px top edge on cards that sweeps once on entry.

Motion tokens

TOKEN	VALUE	USED FOR
dur-micro	150 ms	Hover, press, toggles
dur-short	300 ms	Buttons, chips, small reveals
dur-medium	600 ms	Card and section entrances
dur-long	1200 ms	Hero choreography
ease-out-expo	cubic-bezier(.16, 1, .3, 1)	Default entrance
ease-in-out	cubic-bezier(.65, 0, .35, 1)	Transitions and morphs
stagger	60–80 ms	Lists, grids, words

A 3D hero that degrades gracefully

The hero is one luminous object — a wireframe headset above an ignited orbital ring. It is progressive enhancement: the page is readable before any 3D code loads, and every device gets a version it can run smoothly.

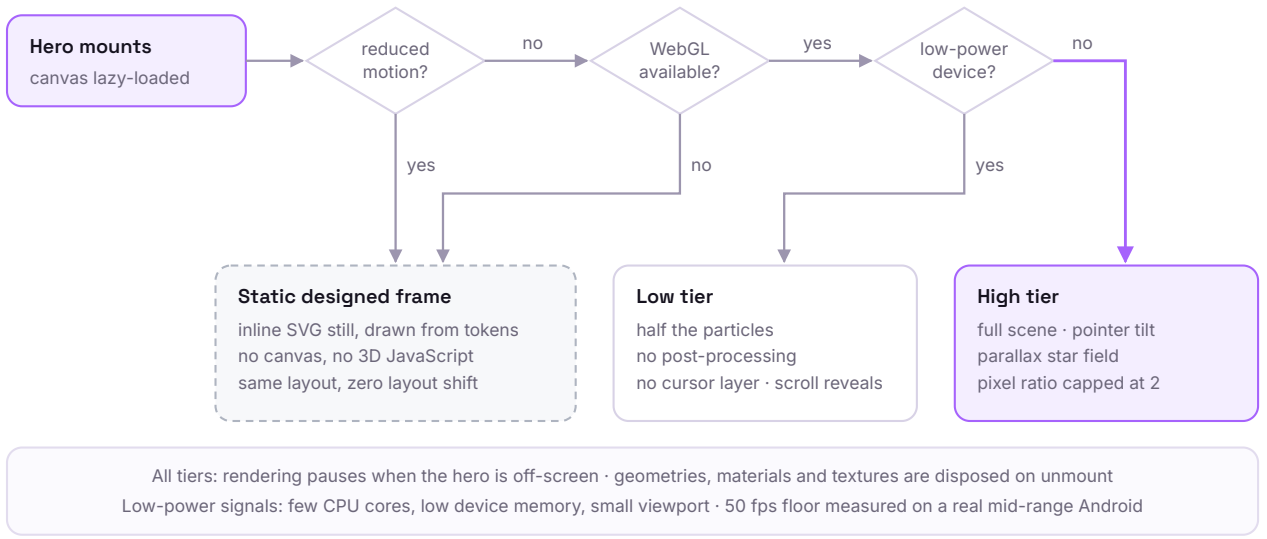


FIG 9 Device-tier selection for the 3D hero.

Hero choreography (first load)

TIME	WHAT HAPPENS
0 – 200 ms	Darkness; the headline and calls to action are already in the page as real text.
200 – 900 ms	The headset outline draws in with a spectrum stroke.
900 – 1500 ms	Particles converge into the wireframe headset; the ring ignites left to right and the horizon fades up.
Idle	The headset floats and tilts toward the pointer; a satellite orbits the ring on a 9 s loop; stars drift at three depths.
Scroll-out	Pinned for 150 vh: the camera pushes through the lens and the ring expands into the next section's background.

Motion guardrails

ONLY
transform and opacity animate outside 3D canvases.

≤ 1.5 s
before content is readable on first load. The intro is skippable and plays once per session.

MAX 2
heavy WebGL components on any page; the hero uses one.

ADMIN
micro-interactions only: no 3D, no ambient motion.

Operating-system "reduce motion" is honoured everywhere: every animated primitive degrades to a simple fade or to no motion at all.

A layout that explains itself

```
sig-reality-spectra/
├─ src/
│  ├─ app/
│  │  ├─ (site)/          public routes, one folder per page
│  │  ├─ admin/           admin routes, noindex
│  │  ├─ api/             upload tokens, nightly job
│  │  ├─ globals.css      design tokens (@theme)
│  │  └─ robots.ts
│  ├─ components/
│  │  ├─ ui/              design-system primitives
│  │  ├─ motion/          animation primitives
│  │  ├─ three/           3D scenes
│  │  ├─ sections/        composed page sections
│  │  └─ admin/           admin UI (shadcn/ui)
│  ├─ lib/
│  │  ├─ data/            repository layer
│  │  ├─ schemas/         Zod schemas
│  │  ├─ auth/            Auth.js config
│  │  ├─ env.ts           server env (server-only)
│  │  ├─ env.public.ts    browser-safe env
│  │  └─ site.ts          sitemap + club config
│  └─ content/            intro, domains, contact
├─ tests/
│  ├─ unit/               Vitest
│  └─ e2e/                Playwright
├─ references/            design inputs, never shipped
├─ docs/                  handover guides
├─ CLAUDE.md              project contract
└─ .env.example           environment contract
```

Permanent engineering rules

01 **Tokens only.** No hard-coded colours, spacing, fonts or durations; one token change updates the whole site.

02 **One data path.** All storage access goes through `src/lib/data`, enforced by lint.

03 **One schema per entity**, shared by the admin form, the server action and the public renderer.

04 **Transform and opacity only**, and reduced motion respected everywhere.

05 **Green gates to finish a stage:** lint, type check, unit tests, build and end-to-end tests.

06 **No public links to `/admin`**, and every admin route is `noindex`.

07 **Never trust the proxy alone** — each server action re-checks the session.

08 **Nothing an admin does can break a page** — layouts survive 0, 1, 3 and 40 items and a 400-character title.

09 **Every write revalidates** the cache tags it affects.

Environment contract. `.env.example` lists every variable the project will ever need, grouped by the stage that introduces it. Variables for features not yet built are shape-checked but optional, so the project always builds; a missing value fails loudly only when the feature that needs it runs.

Proof, not promises

Testing strategy

LAYER	TOOL	WHAT IT PROVES
Static checks	TypeScript strict · ESLint · Prettier	Type safety, typed routes, forbidden imports, consistent formatting.
Unit	Vitest	Schemas accept valid and reject invalid data; repository functions leave no orphaned keys or blobs; sitemap configuration matches the pages on disk and has no per-record routes.
End-to-end	Playwright · Chromium + mobile WebKit	Every page loads; an admin save reaches the public page within 10 s; an ended event leaves the current slot; timeline photos and standalone photos appear in the gallery; the recruitment toggle shows and hides the PDF and form; a sponsorship request reaches the admin inbox; unauthorised access is rejected.
Visual	Playwright snapshots	Timeline, gallery and team grid with 0, 1 and 40 items; very long titles stay inside cards.
Content	Build-time validation	A broken content field fails the build and names the file and field.
Manual	Real devices	Smooth hero on a mid-range Android; keyboard-only and screen-reader walkthroughs.

Budgets and targets

LIGHTHOUSE (MOBILE)	TARGET
Performance	≥ 90 (Home ≥ 85, due to 3D)
Accessibility	≥ 95
Best practices	≥ 95
SEO	100

RUNTIME	TARGET
Frame rate	60 fps laptop · 50+ fps mid-range Android
Layout shift	CLS ≈ 0 from animations
Readable content	≤ 1.5 s after first load
3D code	lazy-loaded, never in the initial bundle
Images	WebP, ≤ 300 KB, correctly sized

Accessibility commitments

WCAG 2.1 AA contrast · Visible focus on every control · Skip link + single main landmark · Focus trapping in modals and lightbox · Alt text required on every poster · Keyboard-operable admin panel · Reduced-motion support · Verified at 360 · 768 · 1280 · 1920 px

Every change previewed, every merge gated

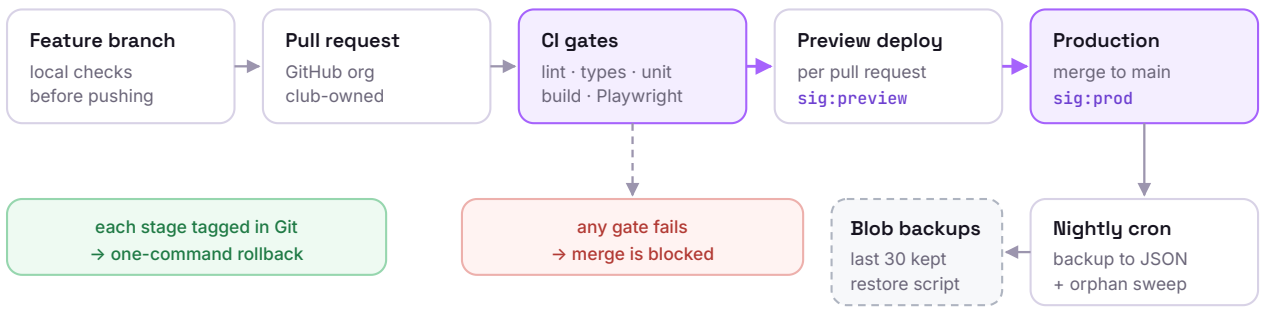


FIG 10 Delivery pipeline. Preview and production use separate Redis key prefixes, so testing can never touch live club data.

Environments

ENVIRONMENT	KEY PREFIX	PURPOSE
Local	`sig:dev`	Development against a personal or shared dev store.
Preview	`sig:preview`	Automatic deploy for every pull request; end-to-end tests run against it.
Production	`sig:prod`	Custom club domain, HTTPS, `www` redirected to the apex domain.

Configuration by feature

FEATURE	VARIABLES
Site	`NEXT_PUBLIC_SITE_URL`
Data	`UPSTASH_REDIS_REST_URL` · `UPSTASH_REDIS_REST_TOKEN` · `REDIS_KEY_PREFIX` · `BLOB_READ_WRITE_TOKEN`
Admin auth	`AUTH_SECRET` · `AUTH_GOOGLE_ID` · `AUTH_GOOGLE_SECRET` · `ADMIN_EMAILS`
Spam protection	`NEXT_PUBLIC_TURNSTILE_SITE_KEY` · `TURNSTILE_SECRET_KEY`
Scheduled jobs	`CRON_SECRET`

Handover package

README — setup in under 10 minutes · ADMIN_GUIDE — for non-developers, with screenshots · CONTENT_GUIDE — copy-paste JSON templates · ARCHITECTURE — this design, kept current · HANDOVER — club accounts and yearly access transfer · GO_LIVE — launch checklist

13 · DELIVERY ROADMAP

Ten stages, each one reversible

Each stage is a self-contained unit of work with its own acceptance checklist. A stage is only complete when every quality gate passes; it is then committed and tagged, so any stage can be rolled back cleanly.

FOUNDATION

1 Foundation and project rules

Scaffold, folder structure, a placeholder route for every page, validated environment contract, test tooling, project rules document.

`gate · every route loads · all checks green`

2 Animation system and 3D hero

Motion tokens, Lenis + GSAP loop, reusable motion primitives, device-tiered hero with static fallback, motion playground.

`gate · 50 fps on Android · reduced motion → still`

EXPERIENCE AND CONTENT

3 Design system and UI shell

Colour, type, spacing and radius tokens; accessible primitives; navbar with the Recruitment call to action; footer; UI showcase.

`gate · AA contrast · one token re-themes all`

4 Static content and page shells

Typed JSON for site info, domains and contact details; Home, About, Recruitment and Contact shells; metadata, sitemap and robots.

`gate · bad content fails the build · Lighthouse`

PLATFORM

5 Data layer and schemas

Event, team, recruitment and sponsorship schemas; Redis/Blob repository with transactions; browser-upload tokens; cache tags; seed script.

`gate · zero orphaned keys or blobs`

6 Admin authentication and shell

Google sign-in with allowlist, route guard, per-action session checks, dashboard, toasts, unsaved-changes guard.

`gate · random account rejected · 401 without session`

FEATURES

7 Events and gallery

Admin: current event, ended events kept for later, recap and photos, add to timeline, plus a gallery manager. Public: current event, timeline, and a gallery of event and club photos by category.

`gate · save → live in 10 s · ended event leaves "current"`

8 Team, recruitment and sponsorships

Team editor with hierarchy and core flag; recruitment toggle with roles PDF and form link; sponsorship form and admin inbox.

`gate · toggle works live · bots rejected`

HARDENING AND LAUNCH

9 Polish, performance, accessibility, SEO

Motion audit, bundle analysis, keyboard and screen-reader pass, social preview images, structured data, security headers.

`gate · Lighthouse targets · social previews`

10 Testing, deployment and handover

Full end-to-end suite, CI with blocked merges, environments, domain, nightly backup and orphan sweep, handover documentation.

`gate · a new admin adds an event from the guide`

What could go wrong, and what we need

Risks and mitigations

RISK	IMPACT	MITIGATION
3D hero is janky on phones	High	Device tiers, static fallback, lazy loading, off-screen pause; measured on a real Android.
Logo unusable on a dark site	High	The current file is a white-background raster; a transparent vector (light, dark, square) is needed first.
Accounts owned by individuals	High	Every service registered to club-owned logins; documented in the handover guide.
Admin data loss	Medium	Nightly JSON backups to Blob, 30 retained, documented restore script; separate preview and production data.
Large photo uploads fail or pile up	Medium	Browser-side WebP compression, direct-to-Blob uploads, 60 photos per event, nightly orphan sweep.

Decisions made

DESIGN

- Orbit Lab hero: rings and one wireframe headset
- Space Grotesk, Inter and JetBrains Mono
- Caption colour `#797F8D` for AA contrast
- React Bits as the only component source
- One WebGL context, no `ogl`, no animated blur

PRODUCT

- Gallery shows event photos and standalone club photos
- One Contact page holds the sponsorship form
- No email alerts: requests live in the admin inbox
- No stat counters on Home in version 1

> **Still open.** Confirm the event categories and the team hierarchy levels, confirm that external hosting is acceptable, and name the next-tenure maintainers.

Inputs needed from the club

NEEDED FOR	INPUTS
Brand	Transparent vector logo (light, dark, square mark) · VIT branding rules
Content	One-line intro · mission · vision · about copy · list of domains · contact details
Events	3–5 past events with photos and a first batch of club photos (consent checked) · one example registration link · poster aspect ratio
Team and recruitment	Team list with photos, positions and hierarchy level · roles PDF · Google Form link · is recruitment open now?
Accounts and launch	Club Vercel project with Upstash Redis and Blob · Google OAuth client · Turnstile keys · official club email · admin email list · domain and DNS access under club ownership